

1. ABAC (Attribute-Based Access Control)

What is it?

ABAC allows or denies requests based on **attributes** of the request:

- User identity
- Resource type
- Action (verb)
- Custom attributes (like labels)

How it works:

- You define a **JSON policy file**.
- The API server reads this file and decides access based on matches.

Pros:

- Very **flexible** (fine-grained control).
- Can enforce complex logic using custom attributes.

Cons:

- **Hard to maintain** (not dynamic).
- Not scalable for large teams.
- Deprecated in most modern Kubernetes setups.

Example:

```
{
  "apiVersion": "abac.authorization.kubernetes.io/v1beta1",
  "kind": "Policy",
  "spec": {
    "user": "alice",
    "namespace": "dev",
    "resource": "pods",
    "readonly": true
  }
}
```

2. RBAC (Role-Based Access Control)

What is it?

RBAC authorizes users based on **roles and bindings**.

Core Concepts:

- **Role**: A set of permissions (verbs + resources).
- **RoleBinding**: Assigns a Role to a user or group in a namespace.
- **ClusterRole / ClusterRoleBinding**: Cluster-wide version.

Pros:

- **Built-in and default** in Kubernetes.
- Easy to manage and scale.
- Works with service accounts and users.

Cons:

- Limited to defined verbs/resources.
- No dynamic conditionals (like time-based access).

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

3. Node Authorization

What is it?

A special, **built-in** authorizer for **Kubelet nodes**.

Purpose:

Allows nodes to access resources **only for the pods they run**.

Used for:

- Letting the node read pod secrets, volumes, or config maps **only** for its scheduled pods.
- Automatically enforced by Kubernetes when using `--authorization-mode=Node`.

Pros:

- Highly secure.
- Prevents nodes from accessing other pods' data.

Cons:

- Limited to internal kubelet use.
- Not used for human or app access.

Summary Table

Feature	ABAC	RBAC	Node
Control Type	Attribute-Based	Role-Based	Node Identity-Based
Configuration	JSON Policy File	YAML Role + Binding	Automatic
Use Case	Flexible logic (legacy)	Most common, scalable	For kubelet access only
Scope	User/resource-specific	Namespaced + cluster-wide	Node-only
Status	Legacy / Not recommended	 Recommended	Auto-managed by K8s
